

Topic

FlagOS Virtual Coding Competition & Live Demo (KernelGen 24 Hour Bounty Challenge)

Jian Tao

Algorithm Engineer, AI Systems Research Group
Beijing Academy of Artificial Intelligence(BAAI)



KernelGen 24 Hour Bounty Challenge

Tao Jian



CONTENTS

GOSIM Paris 2026

Part 01

Introduction

Part 02

Kernel Description

Part 03

Evaluation



01 Introduction

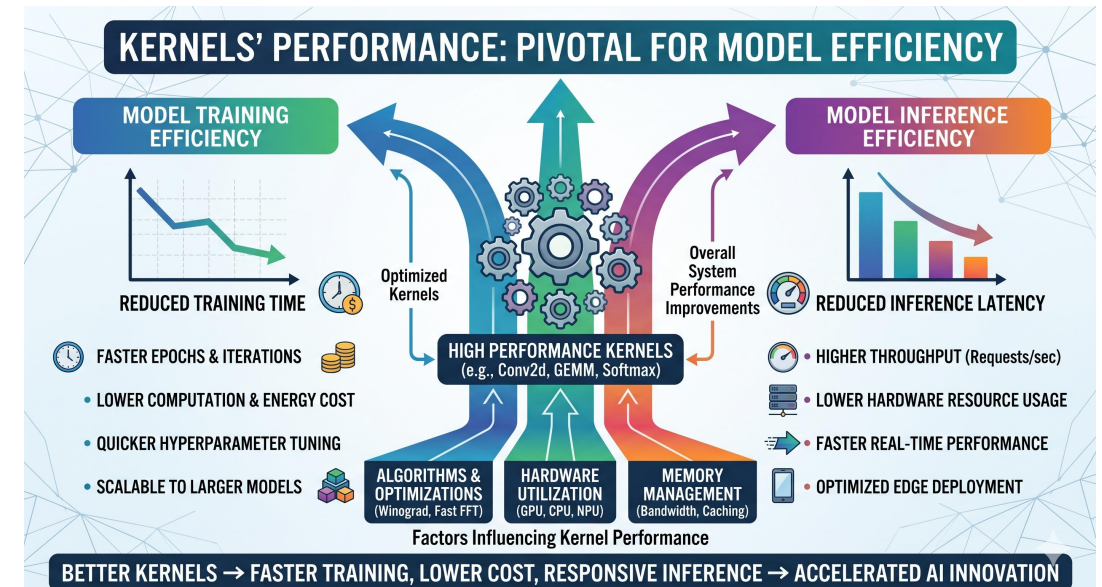


Kernels' **performance** plays a pivotal role in the **training and inference** efficiency of models.

A **24-hour** operator optimization challenge powered by KernelGen, designed for **heterogeneous AI hardware** (GPGPU / NPU).

3 Operators · 3 Leaderboards · Real-Time Ranking · Cash Prizes

<https://jwolpxeehx.feishu.cn/share/base/form/shrcnPa4SXICmklKMrhfkzsSWsd>



Timeline

Submission Phase

Start:

May 5, 2026 — 16:00 (Paris Time)

May 5, 2026 — 22:00 (Beijing Time)

Deadline:

May 6, 2026 — 16:00 (Paris Time)

May 6, 2026 — 22:00 (Beijing Time)

Awards Ceremony

May 6, 2026 | 17:00 (Paris Time)

May 6, 2026 | 23:00 (Beijing Time)

Challenge Format

This competition adopts a **Bounty Mode**:

3 independent operator tasks = 3 leaderboards

Each task is a **real-world** AI kernel optimization problem

Participants submit solutions to compete for ranking

Focus on: **Correctness + Performance**



What You'll Do

For each operator task:

Implement a **high-performance kernel**

Optimize for **specific hardware**
(GPGPU / NPU)

Ensure **correctness** on every
workload we provide

Challenge Format

This competition adopts a **Bounty Mode**:

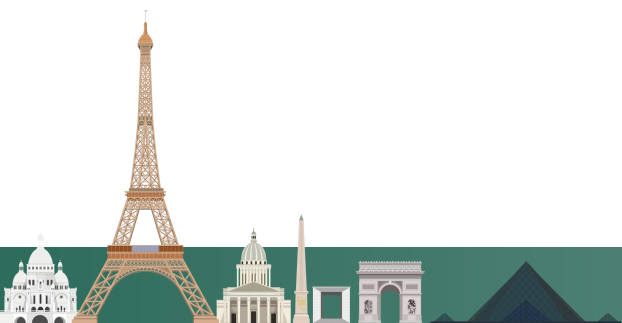
3 independent operator tasks = 3 leaderboards

Each task is a **real-world** AI kernel optimization
problem

Participants submit solutions to compete for
ranking in our website

Focus on: **Correctness + Performance**

<https://kernelgen.flags.io/challenge?lang=en>



02 Kernels' Description



Implement a Top-K Sparse Attention Kernel that computes attention by gathering a subset of Key-Value (KV) pairs, incorporating an Attention Sink mechanism.

Core Challenges

Efficient KV Gathering: Optimizing memory access patterns to collect sparse KV entries.

Sparse Score Computation: Managing non-contiguous attention score calculations.

Attention Sink Integration: Seamlessly blending persistent "sink" tokens into the sparse computation flow.

```
def sparse_attn(q, kv, attn_sink, topk_idx, scale):  
    """  
    Reference sparse attention in pure PyTorch.  
  
    Args:  
        q: [b, m, h, d], bf16  
        kv: [b, kv_len, d], bf16  
        attn_sink: [h], fp32  
        topk_idx: [b, m, topk], int32  
        scale: float  
  
    Returns:  
        o: [b, m, h, d], same dtype as q  
    """  
    b, m, h, d = q.shape  
    topk = topk_idx.shape[-1]  
  
    flat_idx = topk_idx.long().reshape(b, m * topk)  
    gathered_kv = torch.gather(  
        kv,  
        1,  
        flat_idx.unsqueeze(-1).expand(-1, -1, d)  
    ).reshape(b, m, topk, d)  
  
    scores = torch.einsum("bmhd,bmtd->bmht", q.float(), gathered_kv.float()) * scale  
  
    sink = attn_sink[None, None, :, None].expand(b, m, h, 1)  
    scores_with_sink = torch.cat([scores, sink], dim=-1)  
    attn = torch.softmax(scores_with_sink, dim=-1)  
  
    attn_kv = attn[:, :, :, :-1]  
    o = torch.einsum("bmht,bmtd->bmhd", attn_kv, gathered_kv.float())  
    return o.to(q.dtype)
```



Implement a fused kernel for Multi-stream Mixing + Doubly Stochastic Matrix Projection, performing multi-path residual reading and weighted combination in a single memory access.

Core Challenges

Register Pressure Control: Managing high occupancy while storing intermediate states for multiple streams.

Pipeline Hiding for Multi-stream Memory

Access: Overlapping global memory transactions with computation to mask latency.

```
def hc_split_sinkhorn(
    mixes: torch.Tensor,
    hc_scale: torch.Tensor,
    hc_base: torch.Tensor,
    hc_mult: int = 4,
    sinkhorn_iters: int = 20,
    eps: float = 1e-6,
):
    """
    Reference implementation of hc_split_sinkhorn in pure PyTorch.

    Args:
        mixes: [b, s, (2 + hc_mult) * hc_mult], float32
        hc_scale: [3], float32
        hc_base: [(2 + hc_mult) * hc_mult], float32
        hc_mult: number of streams
        sinkhorn_iters: number of Sinkhorn normalization iterations
        eps: small constant for numerical stability

    Returns:
        pre: [b, s, hc_mult]
        post: [b, s, hc_mult]
        comb: [b, s, hc_mult, hc_mult]
    """
    b, s, _ = mixes.shape
    hc = hc_mult

    # split mixes into pre, post, comb raw logits
    pre_raw = mixes[..., :hc]
    post_raw = mixes[..., hc:2 * hc]
    comb_raw = mixes[..., 2 * hc:].reshape(b, s, hc, hc)

    # apply scale + base + activation
```



Implement a fused Mixture-of-Experts (MoE) Kernel that fuses softmax routing, Top-K selection, Gate-Up GEMM, SiLU gated activation, Down GEMM, and weighted reduction into a single operator call.

Core Challenges

Token-to-Expert Dispatch Efficiency: Optimizing the data movement and indexing required to route tokens to their respective experts.

Multi-Expert GEMM Parallelism: Balancing the workload across GPU SMs (Streaming Multiprocessors) when experts receive varying numbers of tokens (load balancing).

SiLU Gating Fusion: Integrating the element-wise gated activation function directly into the GEMM output to minimize memory round-trips.

Weighted Reduction: Efficiently performing the final summation of expert outputs according to their routing probabilities.

```
def fused_moe(hidden_states, w1, w2, score, topk, renormalize=False):
    """
    Reference implementation of fused Mixture-of-Experts in pure PyTorch.

    Computation: softmax routing → top-k expert selection → per-expert
    gate-up GEMM → SiLU gating → down GEMM → weighted reduce.

    Args:
        hidden_states: Tensor [M, K]      - input tokens, bf16
        w1:            Tensor [E, 2*N, K]  - gate+up weights (fused), bf16
                               E = num_experts, N = intermediate_size
        w2:            Tensor [E, K, N]    - down projection weights, bf16
        score:         Tensor [M, E]      - router logits, bf16
        topk:          int                 - number of experts per token
        renormalize:   bool                - whether to renormalize topk weights

    Returns:
        output:       Tensor [M, K]      - output tokens, bf16
    """
    M, K = hidden_states.shape
    E = w1.shape[0]
    N = w2.shape[-1]
    dtype = hidden_states.dtype

    topk_weights = score.softmax(dim=-1, dtype=torch.float32)
    topk_weights, topk_ids = topk_weights.topk(topk, dim=-1)
    if renormalize:
        topk_weights = topk_weights / topk_weights.sum(dim=-1, keepdim=True)
    topk_weights = topk_weights.to(dtype)

    final_out = torch.zeros(M, K, device=hidden_states.device, dtype=dtype)

    for expert_idx in range(E):
        mask = (topk_ids == expert_idx) # [M, topk]
        token_weights = (topk_weights * mask).sum(dim=-1, keepdim=True) # [M, 1]

        x = F.linear(hidden_states, w1[expert_idx]) # [M, 2*N]
        gate = F.silu(x[:, :N])
        x = x[:, N:] * gate # [M, N]
        x = F.linear(x, w2[expert_idx]) # [M, K]

        final_out += x * token_weights

    return final_out
```



03 Evaluation



Compute the average speedup on the 5 chips, 0 score for the wrong result



Ascend



天融智芯
Huawei CoreX



HYGON
中科海光



摩尔线程
MOORE THREADS



METAX
沐曦

	batch	seq_len	kv_len	topk	heads
Prefill	64	1024	1024	128	16
Prefill	64	1024	1280	384	16
Prefill	16	4096	4096	128	16
Prefill	1	16384	16384	128	16
Prefill	1	16384	20480	640	16
Decode	64	1	128	128	16
Decode	64	1	400	392	16
Decode	64	1	640	640	16
Decode	16	1	1408	640	16
Decode	1	1	4480	640	16

	mixes shape (Decode)	Call Percentage	mixes shape (Prefill)	Call Percentage
16k-1k-1	[1, 1, 24]	99.8%	[1, 16384, 24]	0.2%
4k-1k-16	[16, 1, 24]	99.8%	[16, 4096, 24]	0.2%
1k-1k-64	[64, 1, 24]	99.8%	[64, 1024, 24]	0.2%
1k-1-64	[64, 1, 24]	33.3%	[64, 1024, 24]	67.2%

	M	K	N	E	topk
Decode	1	2048	1024	8	2
Decode	64	2048	1024	8	2
Decode	1	2048	1024	64	6
Decode	64	2048	1024	64	6
Prefill	512	2048	1024	8	2
Prefill	4096	2048	1024	8	2



Thanks!



Open Compute for Agentic AI (FlagOS)

